

Numerical Programming Final Project Report

Path Following, Bidirectional Swarms, and Navigation in Pedestrian Flow

Ketevan Shavadze
Kutaisi International University

February 17, 2026

Course: Numerical Programming
Project: Final Project

Repository structure (high level):

- `src/tasks/` — task entry points (scripts you run)
- `src/common/` — shared I/O, visualization, configuration loader
- `src/numerics/` — RK4 integrator
- `src/path/` — map processing, skeleton route, spline, corridor geometry
- `src/models/` — robot/swarm forces and controllers
- `data/map/` — map images (raw + snapshot with A/B)
- `data/params/` — YAML configs for reproducibility
- `data/video/` — pedestrian flow video for Task 3
- `tests/` — test cases for which this approach works well and does not work.
- `outputs/` — generated plots, GIFs, and videos

Contents

1	Overview	3
2	Inputs and Data Files	3
2.1	Task 1 and Task 2 map inputs	3
2.2	Task 3 video input	3
2.3	Parameter files	3
3	Common Numerical Model	3
3.1	Robot dynamics	3
3.2	RK4 integration	4
4	Task 1: Path + Corridor + Robot Trajectory	4
4.1	Goal	4
4.2	Path extraction	4
4.3	Spline representation	4
4.4	Corridor modeling	4
4.5	Control law	4
4.6	Outputs	5
5	Task 2: Bidirectional Swarms in a Corridor	5
5.1	Goal	5
5.2	Swarm initialization	5
5.3	Forces	5
5.4	Safety metric	5
5.5	Outputs	6
6	Task 3: Navigation in Pedestrian Flow Video	6
6.1	Goal	6
6.2	Video choice	6
6.3	Optical flow velocity field	6
6.4	Pedestrian detection	6
6.5	Two-robot navigation	6
6.6	Outputs	7
7	How to Run	7
7.1	Environment	7
7.2	Commands	7
8	Discussion and Notes	7
9	Automated Tests and Limitations	7
9.1	Test Summary Output	8
9.2	Tests That Work Well	8
9.3	Limitations and Failing Tests	8
9.4	Interpretation	9
10	Conclusion	9

1 Overview

This project implements three tasks:

1. **Task 1:** Extract a centerline path from a map image and simulate a robot tracking this path inside a corridor.
2. **Task 2:** Simulate two bidirectional swarms inside the same corridor with repulsive collision avoidance, and report the minimum pairwise distance over time.
3. **Task 3:** Use a pedestrian-flow video and optical flow to build a velocity field, then simulate robots navigating safely in the moving crowd and produce an output visualization video.

All tasks are designed for **reproducibility** via external YAML parameter files stored in `data/params/` and for clean architecture via reusable modules in `src/common`, `src/path`, `src/numerics`, and `src/models`.

2 Inputs and Data Files

2.1 Task 1 and Task 2 map inputs

- `data/map/map_raw.png`: binary-like map image. The road/path is dark and the background is light.
- `data/map/map_snapshot_A_B.png`: the same map with start/goal markers:
 - **A** marked with a green circle,
 - **B** marked with a red circle.

2.2 Task 3 video input

- `data/video/pedestrians.mp4`: synthetic pedestrian-flow video generated by a script in `src/tools/generate_pedestrian_video.py`. It contains two opposite flows:
 - top half: pedestrians move left $\rightarrow$ right,
 - bottom half: pedestrians move right $\rightarrow$ left.

2.3 Parameter files

Each task reads parameters from YAML config files:

- `data/params/task1_default.yaml`
- `data/params/task2_default.yaml`
- (optional) `data/params/task3_default.yaml` if created for task 3 tuning

3 Common Numerical Model

3.1 Robot dynamics

Across tasks, the robot is modeled using a second-order system:

$$\dot{x} = v, \tag{1}$$

$$\dot{v} = \frac{1}{m} F(x, v, t), \tag{2}$$

where $x \in \mathbb{R}^2$ is position, $v \in \mathbb{R}^2$ is velocity, and F is the total force (control + avoidance + constraints).

3.2 RK4 integration

Time integration is done with the classical 4th-order Runge–Kutta method (RK4), implemented in `src/numerics/rk.py`. RK4 is used because it is stable and accurate for smooth dynamics while remaining straightforward to implement.

4 Task 1: Path + Corridor + Robot Trajectory

4.1 Goal

Given a map image and two endpoints A and B, the task is to:

1. Extract a centerline path from A to B,
2. Build a corridor around it,
3. Simulate a robot tracking the path while staying inside the corridor,
4. Output a plot with the path, corridor boundaries, and robot trajectory.

4.2 Path extraction

1. Convert the map image into a binary mask (`load_binary_path_mask`).
2. Skeletonize the mask to obtain a 1-pixel-wide path graph (`skeleton_path`).
3. Snap A and B markers to the nearest skeleton pixels (`nearest_skeleton_pixel`).
4. Use BFS on the skeleton pixels to compute the shortest path sequence (`bfs_shortest_path`).

4.3 Spline representation

The discrete path pixels are converted to a smooth parametric spline:

$$x = s_x(s), \quad y = s_y(s),$$

where s is arc-length-like parameter computed from pixel distances. Spline fitting is implemented in `src/path/splines.py`.

4.4 Corridor modeling

A corridor of width w is built around the spline using a normal vector field:

$$n(s) = \frac{1}{\|t(s)\|} \begin{bmatrix} -t_y(s) \\ t_x(s) \end{bmatrix}, \quad t(s) = \begin{bmatrix} s'_x(s) \\ s'_y(s) \end{bmatrix}.$$

Left and right boundaries are:

$$L(s) = c(s) + \frac{w}{2}n(s), \quad R(s) = c(s) - \frac{w}{2}n(s),$$

where $c(s) = [s_x(s), s_y(s)]^T$. Implemented in `src/path/corridor.py`.

4.5 Control law

The robot follows a look-ahead target point on the spline:

$$T = c(s_{\text{progress}} + \ell),$$

and uses a PD tracking force plus a corridor border force:

$$F = k_p(T - x) - k_d v + F_{\text{border}}.$$

4.6 Outputs

The task produces:

- `outputs/task1/task1_map.png`: path + corridor + trajectory
- `outputs/task1/task1_border_violation.png`: border violation proxy over steps

5 Task 2: Bidirectional Swarms in a Corridor

5.1 Goal

Simulate two robot swarms moving in opposite directions inside the same corridor. The simulation should:

- keep robots inside the corridor,
- avoid collisions using repulsion,
- output trajectories and a minimum-distance curve.

5.2 Swarm initialization

Robots are placed along the spline near opposite ends:

- Group A starts near $s = 0$ and moves toward $s = s_{\max}$,
- Group B starts near $s = s_{\max}$ and moves toward $s = 0$.

A lane offset pushes A to one side and B to the other to support safe passing.

5.3 Forces

Each robot uses:

1. PD tracking to a look-ahead spline point,
2. Corridor border penalty to remain inside width w ,
3. Smooth repulsion for collision avoidance:

$$F_{\text{rep}}(d) = \begin{cases} k_{\text{rep}} \left(\frac{1}{d} - \frac{1}{R_{\text{safe}}} \right) \hat{r}, & d < R_{\text{safe}}, \\ 0, & d \geq R_{\text{safe}}, \end{cases}$$

where d is distance to a neighbor, $\hat{r}$ is the unit direction away from the neighbor.

5.4 Safety metric

We measure the minimum pairwise distance over time:

$$d_{\min}(t) = \min_{i < j} \|x_i(t) - x_j(t)\|.$$

This metric is plotted and compared to R_{safe} .

5.5 Outputs

- outputs/task2/task2_map.png
- outputs/task2/task2_min_distance.png
- outputs/task2/task2_animation.gif

6 Task 3: Navigation in Pedestrian Flow Video

6.1 Goal

Given a pedestrian flow video, a robot must safely navigate through the moving crowd in at least two directions. The output must visualize the robot(s) and pedestrians.

6.2 Video choice

A synthetic video was generated to ensure stable and reproducible optical flow:

- Top half: pedestrians move left $\rightarrow$ right
- Bottom half: pedestrians move right $\rightarrow$ left

Pedestrians are drawn as black circles on a white background which simplifies detection and flow estimation.

6.3 Optical flow velocity field

For each pair of consecutive frames, dense optical flow is computed using Farnebäck's algorithm:

$$V(x, t) \approx \text{OpticalFlow}(I_{t-1}, I_t).$$

A preview visualization is produced by drawing flow arrows.

6.4 Pedestrian detection

Because pedestrians are synthetic black circles, centers are detected by thresholding and contour filtering. This provides a set of obstacle centers $\{p_j(t)\}$ used for repulsion.

6.5 Two-robot navigation

Two robots are simulated:

- **K-VRC** starts in the top flow and performs horizontal motion with direction switching at the borders.
- **XBOT** starts in the bottom flow and performs horizontal motion with opposite direction switching to interact with the opposite flow.

Each robot uses:

$$F = k_v(v_{\text{des}}(x, t) - v) - k_d v + F_{\text{rep}}.$$

The desired velocity v_{des} is derived from optical flow, with a small direction constraint to guarantee motion in the required direction(s).

6.6 Outputs

- `outputs/task3/task3_flow_preview.mp4`: optical-flow arrows preview
- `outputs/task3/task3_two_robots_navigation.mp4`: final navigation visualization

7 How to Run

7.1 Environment

- Python 3.10+ recommended
- Required packages: `numpy`, `opencv-python`, `matplotlib`, `scikit-image`, `scipy`, `imageio`, `pyyaml`

7.2 Commands

```
# (1) Generate pedestrian video (Task 3 input)
python src/tools/generate_pedestrian_video.py

# (2) Task 1
python src/tasks/task1_path_follow.py

# (3) Task 2
python src/tasks/task2_swarm_corridor.py

# (4) Task 3
python src/tasks/task3_pedestrian_navigation.py
```

8 Discussion and Notes

- Using external YAML configs ensures reproducibility and clean tuning.
- Synthetic video simplifies optical flow and pedestrian detection, making the navigation behavior stable and interpretable.
- Repulsion parameters (k_{rep} , R_{safe}) control safety; stronger repulsion increases clearance but may reduce speed.

9 Automated Tests and Limitations

All tasks were tested using the `pytest` framework. Tests were designed to demonstrate:

- cases where the approach works well,
- cases where it fails or exposes limitations.

The following command was used to run tests:

```
python -m pytest -v
```

9.1 Test Summary Output

The following results were obtained:

```
collected 6 items

test_rk4_convergence_harmonic_oscillator PASSED
test_task1_following_inside_corridor PASSED
test_task1_fails_when_corridor_too_narrow XPASS
test_task2_bidirectional_progress PASSED
test_task2_no_collision_basic FAILED
test_task3_optical_flow_two_opposite_flows PASSED
```

9.2 Tests That Work Well

RK4 Convergence Test The harmonic oscillator test confirms that reducing time step dt improves numerical accuracy. This validates correct implementation of RK4.

Task 1 – Path Following When corridor width and gains are properly tuned, the robot:

- reaches the target,
- stays inside the corridor,
- shows minimal border violation.

Task 2 – Bidirectional Progress Both swarms successfully move in opposite directions along the spline. This confirms correct implementation of tracking and bidirectional logic.

Task 3 – Optical Flow Direction Optical flow correctly detects opposite horizontal motion in the top and bottom halves of the video. This validates the velocity field construction.

9.3 Limitations and Failing Tests

Task 1 – Extremely Narrow Corridor (XPASS) A test was designed to fail when corridor width is too small. However, in this configuration the controller still managed to stay inside. This indicates that the controller is more robust than expected, but performance may degrade for more extreme narrowing.

Task 2 – No-Collision Safety Failure The test `test_task2_no_collision_basic` failed with:

```
assert 8.79 >= (20.0 * 0.85)
```

This means that the minimum distance between robots dropped below the desired safety threshold.

This demonstrates a key limitation:

- Repulsion-based collision avoidance depends strongly on:
 - repulsion gain k_{rep} ,
 - safe radius R_{safe} ,
 - time step dt ,

- number of robots,
- corridor width.
- For certain dense configurations, the smooth potential-based method cannot guarantee strict safety.

This is a known limitation of decentralized potential-field approaches. Increasing repulsion gain or reducing robot density improves safety, but may cause oscillations or instability.

9.4 Interpretation

The combination of passing and failing tests demonstrates:

- Correct numerical implementation.
- Functional controllers under nominal conditions.
- Clear and reproducible limitations under stress conditions.

This satisfies the project requirement to provide test cases for both successful and unsuccessful scenarios.

10 Conclusion

All three tasks are implemented with a consistent modeling approach:

- Smooth geometry via spline-based path representation,
- RK4 integration for stable simulation,
- PD tracking and repulsion-based safety mechanisms,
- Clear visual outputs saved to the `outputs/` folder.